

1. What is Axon?

Axon is a local-first, OTLP-native observability tool that lets developers monitor, debug, and visualise their LangChain AI agent executions in real-time. All tracing data is stored on your own machine — no cloud account required, no sensitive prompts leaving your environment.

Package	Role	Default Port
@axon-ai/cli	Command-line tool — init, start, stop, status	—
@axon-ai/backend	Express REST + WebSocket server + SQLite storage	:3000
@axon-ai/dashboard	React web UI — graph, inspector, project switcher	:4000

Axon

2. Key Features

AI Agent Observability Platform

■ Real-time Trace Visualisation

Every LLM call, tool invocation, and chain step appears instantly as your agent runs.

■ Interactive Graph View

Zoomable, pannable DAG showing the full execution flow — nodes connected by data edges.

■ Node Detail Inspection

Click any node to view inputs, outputs, token counts, latency (ms), and estimated cost (\$).

■ Project Organisation

Group traces by project name; switch between projects in the left sidebar without restarting.

■ Local SQLite Storage

All data is persisted locally in traces.db — no external services, no subscriptions.

■ WebSocket Live Updates

Dashboard subscribes to the backend via WebSocket — zero-lag updates as events arrive.

■ REST API for History

Query past traces via HTTP endpoints (GET /traces, GET /traces/:id) for scripting or CI.

■ One-command Setup

axon-ai start launches the backend AND dashboard and opens your browser automatically.

■ Docker Support

Run the full stack with make start using Docker Desktop — no Node.js install needed.

3. Architecture

Axon follows a simple three-tier pipeline. The tracer library hooks into your LangChain callbacks and ships events to the backend over HTTP. The backend persists them to SQLite and broadcasts them over WebSocket to any connected dashboard instances.

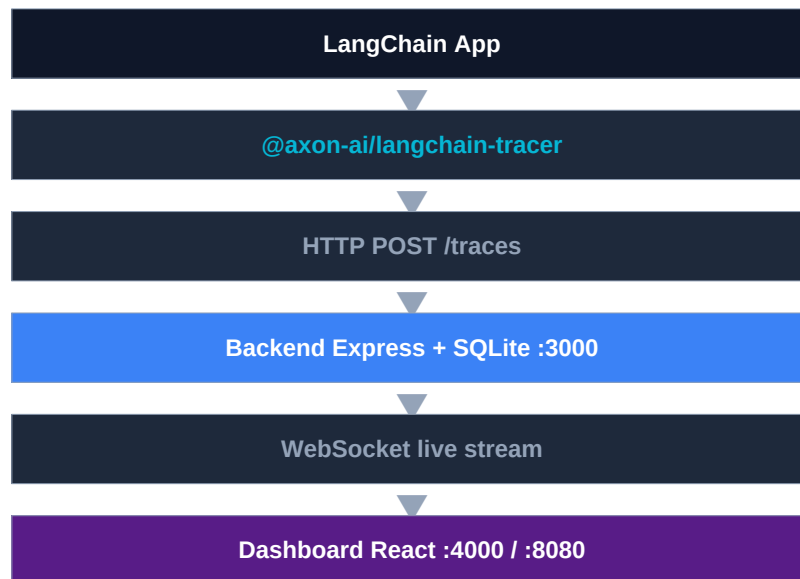


Figure 1 — Data flow from LangChain application to Axon Dashboard

4. User Journey — Getting Started

Step 1 Install the CLI

```
npm install -g @axon-ai/cli
```

Install the Axon CLI globally. Requires Node.js 18+.

Step 2 Initialise your project

```
axon-ai init --project my-app
```

Run inside your LangChain project root. Creates `.axon-ai/config.json` and initialises the database.

Step 3 Start Axon services

```
axon-ai start
```

Launches the backend (port 3000) and dashboard (port 4000) and opens your browser automatically.

Step 4 Integrate the tracer

```
npm install @axon-ai/langchain-tracer
```

Add the tracer to your LangChain project. Import `AxonTracer` and pass it as a callback to your chain or agent.

Step 5 Run your agent

```
node my_agent.js
```

Execute your LangChain application as normal. The tracer silently posts span events to the backend.

Step 6 View live traces

```
http://localhost:4000
```

Open (or switch to) the dashboard. Nodes appear on the graph canvas as each step completes.

Step 7 Inspect a node

```
Click any node on the graph
```

The right-hand Detail Panel shows: inputs, outputs, token count, latency, and estimated USD cost.

Step 8 Debug & iterate

Compare runs side-by-side in the sidebar

Use the timeline metadata to pinpoint slow steps, unexpected tool calls, or runaway token usage.

5. CLI Commands Reference

Command	Description	Example
<code>axon-ai init</code>	Initialise Axon in your project	<code>axon-ai init --project my-app</code>
<code>axon-ai start</code>	Start backend + dashboard	<code>axon-ai start --port 4000</code>
<code>axon-ai status</code>	Check if services are running	<code>axon-ai status</code>
<code>axon-ai stop</code>	Stop all Axon services	<code>axon-ai stop</code>
<code>axon-ai version</code>	Show version information	<code>axon-ai version</code>

6. Dashboard UI Guide

Left Sidebar

Lists all projects configured with `axon-ai init`. Within each project, every historical trace run is shown in reverse-chronological order. Click a trace to load its graph. A live indicator pulses when the backend is actively receiving events from a running agent.

Main Graph Canvas

Renders the full execution DAG. Nodes represent individual spans (LLM calls, tool calls, chain starts/ends). Edges represent parent-child relationships. Use scroll-to-zoom and drag-to-pan to navigate large graphs. New nodes animate in as they arrive over WebSocket.

Right Detail Panel

Displays metadata for the selected node: raw input prompt, model response, token breakdown (prompt / completion / total), latency in milliseconds, estimated cost in USD, and any custom metadata emitted by your chain.

Node Colour Legend

Colour	Node Type	Meaning
Blue	LLM Call	An interaction with a Large Language Model (e.g. GPT-4, Claude)
Green	Tool Call	An external tool invocation (web search, code exec, API call)
Purple	Chain / Agent	A complete agent run or sub-chain — wraps all child nodes

7. Configuration

Axon stores project settings in `.axon-ai/config.json` at your project root. This file is created automatically by `axon-ai init` and can be edited manually.

```
{
  "project": "my-app",
  "backend": {
    "port": 3000,
    "host": "localhost"
  },
  "dashboard": {
    "port": 4000
  }
}
```

Docker Configuration

Service	URL
Dashboard	http://localhost:8080
Backend	http://localhost:3000

To override ports when using the CLI pass flags inline:

```
axon-ai start --port 5000 --dashboard-port 8000
```

8. Prompt Engineering Tips

■ Pro Tips

- Use Axon to identify which prompts are costing the most tokens — sort runs by token count in the sidebar.
- Compare trace graphs before and after prompt changes to verify that your refactor reduced latency.
- Watch tool call frequency to detect agent loops — if a tool node repeats 10+ times, the agent is stuck.
- Inspect the Detail Panel's raw output to catch hallucinations early — before they reach production.
- Use the project switcher to compare the same query across different model configurations side-by-side.

9. Quick-start Cheat Sheet

Task	Command
Install CLI	<code>npm install -g @axon-ai/cli</code>
Init project	<code>axon-ai init --project my-app</code>
Start (CLI)	<code>axon-ai start</code>
Start (Docker)	<code>make start</code>
Check running status	<code>axon-ai status</code>
Stop services	<code>axon-ai stop</code>

View dashboard	http://localhost:4000
View Docker dashboard	http://localhost:8080
Reset Docker DB	<code>docker volume rm axon_axon-data</code>
Run tests	<code>cd test-langchain-project && npm run test:comprehensive</code>